Phase 1 — Literature Review, Datasets, Solution Concept

Julia Kruk and **Nadia Serafin** and **Tomasz Żywicki**

Abstract

1 State of the Art

1.1 Retrieval Augmented Generation (RAG)

The concept of Retrieval Augmented Generation (RAG) was proposed by the authors of P. Lewis et al. Their approach combined parametric memory with non-parametric memory for the purpose of performing natural language processing (NLP) tasks requiring factual knowledge. The authors pointed out the limitations of pre-trained language models which store the knowledge within the models' parameters. Three main problems included difficulties in updating and expanding this knowledge, limited interpretability of the generated output, and the presence of hallucinations, i.e. the generation of information which is inconsistent with the facts. The proposed RAG was designed to reduce these problems by providing an external knowledge source for the model in a text generation task.

The architecture proposed by P. Lewis et al. consists of two main components: an information retrieval module and a text generator. The retriever, based on the Dense Passage Retriever (DPR) method, retrieves document fragments that match the user's query, using a vector index created on the basis of Wikipedia articles. The retrieved fragments provide additional context for the generator, which, in the presented solution, is a pre-trained sequence-to-sequence BART model. The authors proposed two architecural variants: RAG-Sequence, in which a single document determines the generation of the entire sequence, and RAG-Token, which allows different documents to be taken into account when generating individual tokens.

The effectiveness of the proposed approach was evaluated on four NLP tasks requiring factual knowledge: question answering (QA; open-domain question answering and abstractive question answering), question generation, and fact verification. In the open-domain QA task on the Natural Questions dataset (Kwiatkowski et al.), the RAG-Sequence variant achieved 44.5% Exact Match, outperforming DPR (41.5%) and Retrieval-Augmented Language Model Pre-Training (40.4%). In generative tasks, RAG outperformed Bidirectional and Auto-Regressive Transformers (BART) (M. Lewis et al.), both on the Microsoft MAchine Reading COmprehension (MS MACRO) (Bajaj et al.) dataset (44.2 vs. 41.6 BLEU-1) and in Jeopardy question generation using the SearchQA dataset (Dunn et al.), achieving 22.2 vs. 19.7 Q-BLEU-1. RAG also outperformed BART in fact verification on the Fact Extraction and VERification dataset (Thorne et al.), achieving 72.5% vs. 64.0% accuracy, although it did not achieve the best result among all compared systems. Further experiments demonstrated that the method of information retrieval influences the system's final effectiveness, and that the knowledge available to the model can be updated by replacing the external document index without the need to retrain the model.

The development of RAG has led to increasingly sophisticated architectures that go beyound the original retrieval and generation approach. Gao provides a survey of RAG methods, distinguishing three RAG paradigms: the Naive RAG, the Advanced RAG, and the Modular RAG.

Naive RAG is based on a simple workflow that consists of indexing, retrieval and generation. Documents are divided into smaller fragments, converted into vector representations

and stored in an index. For each query, the most relevant chunks are retrieved and provided to a large language model (LLM) as additional context. However, this approach faces limitations related to inaccurate retrieval, irrelevant or redundant information, and the generation of answers unsupported by the retrieved context.

Advanced RAG addresses these limitations by introducing optimizations before and after retrieval. Pre-retrieval techniques include query rewriting, query expansion, improved document segmentation, and metadata enrichment. Post-retrieval techniques, such as reranking and context compression, aim to improve relevance and quality of information passed to the generator.

Modular RAG further extends this architecture by introducing two main innovative developments: the addition of specialized modules and the processing workflows. New components, such as Search, Memory, and Routing modules, extend the system's capabilities by enabling access to different knowledge sources, memory-guided retrieval, and query-dependent search paths. What is more, existing modules can be rearranged or invoked repeatedly rather than following a fixed retrieval-generation sequence.

The effectiveness of RAG depends not only on the quality of retrieved information but also on the ability of the LLM to use it. Research Joren et al. investigated this relationship by introducing the concept of sufficient context, defined as context containing enough information to formulate a credible answer to a given question.

The authors evaluated several LLMs including Gemini 1.5 Pro, GPT 4o, Claude 3.5 Sonnet, and Gemma 27B on three datasets: FreshQA (Vu et al.), Musique (Trivedi et al.), and HotpotQA (Yang et al.). The results showed that answer correctness varied considerably across LLMs, but all models generally achieved higher accuracy when provided with sufficient context. However, even with sufficient context, models sometimes generated incorrect answers, while insufficient context more often led to hallucinations and abstentions. The authors also suggested that optimizing retrieval methods could increase the proportion of cases with sufficient context, although investigating this relationship was beyond the scope of their study.

1.2 RAG Frameworks

Many RAG implementations frameworks have been introduced. FlashRAG (Jin et al.) is a framework providing 16 RAG algorithms that are evaluated within a unified framework; supporting multimodal RAG deployments and containing a visual web interface. The standard algorithm with advanced retrievers and generators, proved to be a strong baseline, showing robust performance on 6 datasets (NQ, TriviaQA, HotpotQA, 2Wiki, PopQA, and WebQA). Another solution – LongRAG (Jiang et al.) – is comprised of two components: the Long Retriever and the Long Reader. Compared to traditional RAG, which operates on a large number of short retrieval units, LongRAG operates on long retrieval units, with only a few (typically fewer than 10) are fed into the reader. Another work, Fan et al., proposed MiniRAG that reimagines the information retrieval and generation pipeline with a focus on extreme simplicity and computational efficiency. MiniRAG consists of two key components: heterogeneous graph indexing which creates a semantic-aware knowledge representation, and lightweight graph-based knowledge retrieval, which enables efficient and accurate information retrieval. Another popular RAG framework is LangChain Chase. Its retrieval pipeline involves loading documents, splitting them into chunks, generating vector embeddings, and storing them in a vector database. At query time, relevant chunks are retrieved using similarity search and provided as context for answer generation. LangChain supports different embedding models and vector stores, allowing retrieval components to be configured according to system requirements. Similarly, LLamaIndex Liu is a framework for developing RAG applications, with a particular focus on integrating external data sources. It provides components for loading, indexing, storing, and querying data. Documents are divided into smaller units called nodes, which can be represented as vector embeddings and stored in a vector store. During retrieval, LlamaIndex uses retrievers to identify relevant nodes based on a

user query. It also supports more advanced retrieval strategies, including hybrid retrieval, routing between retrievers, and reranking of retrieved results.

1.3 Datasets for RAG Evaluation

The task of question answering (QA) is an objective way to test the reasoning ability of a RAG system. HotpotQA (Yang et al.) is a QA dataset that requires reasoning over multiple documents and is collected by crowdsourcing based on Wikipedia articles. Another QA dataset, TriviaQA Joshi et al., is a dataset of 650K question-answer-evidence triples, with questions created by trivia enthusiasts.

1.4 RAG metrics

The article Es et al. introduced Retrieval Augmented Generation Assessment (RAGAs), a framework for automated reference-free evaluation of RAG systems. The authors proposed three metrics: Faithfulness, Answer Relevance, and Context Relevance, all of which rely on LLMs. Faithfulness measures whether generated answers are supported by the retrieved context. Answer Relevance assesses how well answers address the original questions. Context Relevance evaluates the relevance of retrieved information. To validate these metrics, the authors introduced the WikiEval dataset, constructed from 50 Wikipedia pages. To assess the proposed evaluation framework, the authors compared RAGAs' assessments with human judgements. RAGAs agreed with human preferences in 95% of comparisons for Faithfulness, 78% for Answer Relevance, and 70% for Context Relevance, outperforming two baseline evaluation approaches : GPT Score and GPT Ranking.

In the article by Thakur et al. Benchmarking Information Retrieval (BEIR) was proposed a heterogeneous benchmark designed to evaluate the zero-shot information retrieval ability. The benchmark consists of 18 datasets covering nine retrieval tasks and was used to assess ten different retrieval methods. The BEIR framework provides several traditional Information-Retrieval (IR) metrics, including Precision, Recall, Mean Average Precision (MAP), Mean Reciprocal Rate (MRR) and Normalised Cumulative Discount Gain (nDCG@k, Wang et al.) for any top-k hits. However, the authors selected nDCG@10 as the primary evaluation metric as it provides a suitable metric for evaluating retrieval performance with both binary and graded relevance judgements. Specifically on the HotpotQA test set, the BM25 + Cross-Encoder (BM25 + CE) retrieval method achieved an nDCG@10 score of 0.707, outperforming the standard BM25 retrieval method, which scored 0.603.

1.5 Previous work

Brown et al. conducted a systematic literature review of RAG techniques, evaluation methods, and existing challenges. The study aimed to provide a comprehensive overview of the development of RAG systems, synthesise empirical evidence regarding their effectiveness, and identify research gaps. Following the PRISMA 2020 guidelines, the authors examined publications from January 2020 to May 2025, collected from five scientific databases: ACM Digital Library, IEEE Xplore, Scopus, ScienceDirect, and DBLP. From an initial set of 4721 records, 128 studies met the established inclusion criteria and were selected for further analysis.

The reviewed studies covered a variety of application domains, with knowledge-intensive tasks (27.3%) and open-domain question answering (15.6%) being the most frequently investigated. Software engineering and medical applications also represented significant areas of research. The authors organised the existing approaches into several interconnected areas, including retrieval mechanisms, vector databases, document chunking, embedding models, and generation techniques. Particular attention was given to the evolution from conventional single-pass RAG architectures towards more advanced solutions incorporating hybrid retrieval, graph-based indexing, query reformulation, reranking, and iterative retrieval. These developments aim to improve the relevance of retrieved information, provide better contextual grounding, and reduce the limitations of traditional retrieval-generation pipelines.

The review also examined the evaluation practices used to assess RAG systems. The authors identified three main groups of evaluation methods: automatic generation metrics, retrieval metrics, and human or LLM-based

Table 1: A comparison of the results of different methods on selected datasets.

Dataset	Metric	FlashRAG (standard) Jin et al.	FlashRAG (AAR) Jin et al.	LongRAG Jiang et al.	MiniRAG (gpt-4o-mini) Fan et al.
NQ	EM	35.1	30.1	62.7	-
TriviaQA	EM	58.8	56.8	-	-
HotpotQA	F1	35.3	33.4	64.3	-
2Wiki	F1	-	-	-	-
PopQA	F1	-	-	-	-
WebQA	EM	-	-	-	-
LiHuaWorld	Acc %	-	-	-	54.1

assessments. Commonly used generation metrics include Exact Match (EM), Accuracy, and F1, while retrieval performance is typically evaluated using measures such as Precision@k, Recall@k, MRR, and nDCG. The authors emphasised that evaluating only the correctness of generated answers is insufficient, as it does not necessarily indicate whether the retrieved evidence is relevant or whether the generated response is actually grounded in that evidence.

Finally, they identified several persistent challenges affecting the practical deployment of RAG systems. These include retrieval quality, the propagation of errors between pipeline components, computational costs and latency, outdated or domain-specific knowledge, and security threats such as retrieval-side data poisoning and prompt injection. The authors concluded that future research should focus on more comprehensive evaluation frameworks that jointly consider answer quality, retrieval effectiveness, reliability, computational efficiency, and security. They also highlighted the need for more standardised benchmarks to enable meaningful comparisons between different RAG architectures.

2 Datasets and Their Status

The dataset used in the experiment will be a small (150–200 questions) subset of the HotpotQA dataset.

2.1 HotpotQA

- **Status:** READY
- **Size:** 112 779 items in total; a fixed subset of approx. 150–200 questions will be sampled for the evaluation benchmark.
- **Label definition:** The ground truth consists of a textual answer span and a list of supporting facts (specific sentences from the context) required for multi-hop reasoning.
- **Class balance:** Question types are naturally imbalanced, with 'bridge' reasoning questions constituting approx. 80% and 'comparison' questions approx. 20%.
- **Licence:** CC BY-SA 4.0
- **How obtained:** Downloaded from the official repository.

3 Solution Concept and Experimental Plan

The evaluation will compare traditional frameworks (LangChain, LlamaIndex) with other RAG systems (MiniRAG, FlashRAG, LongRAG). The experiments will be conducted only on the retriever part of the RAG architecture.

3.1 Experimental Setup

The experiments will be conducted on a benchmark of 150–200 multi-hop questions sampled from the HotpotQA distractor dataset. The experimental setup will hold the following components strictly constant across all implementations: parsing algorithms, text chunking strategies, the embedding model, and retrieval depth (top-k). The diagram of the system is shown in Figure 1.

3.2 Metrics and Deliverables

The evaluation will rely on retrieval-related metrics:

- **Retrieval Quality:** Recall@k, MRR, and Context Hit Rate.
- **Efficiency:** Retrieval latency, indexing time, and API/token costs.

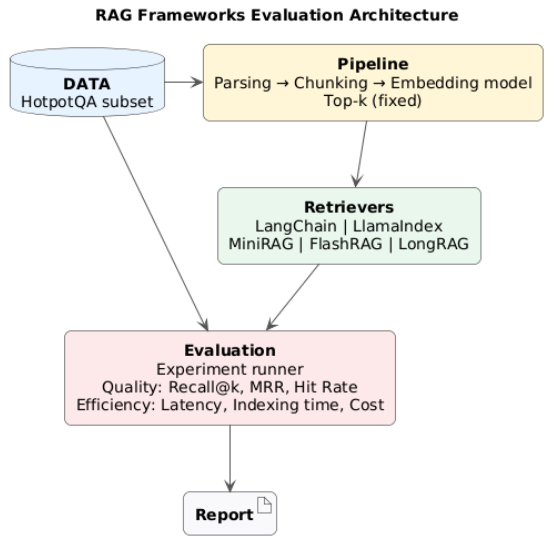


Figure 1: Experimental pipeline architecture.

The core success deliverables for this project are:

1. A reproducible evaluation repository containing isolated retriever implementations across five frameworks with a fixed embedding model.

2. A comparative analysis report attributing performance and latency differences to the underlying framework architectures.

3.3 Week-by-Week Plan

Week-by-Week project plan is included in the Table 2.

4 Data Statement

The data used in this study comes from https://hotpotqa.github.io. It is distributed under the CC BY-SA 4.0 licence. This dataset does not contain personal data.

Works Cited

Lewis, Patrick, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". *Advances in Neural Information Processing Systems*. Edited by H. Larochelle et al., Curran Associates, 2020, pp. 9459–74, proceedings . neurips . cc / paper _ files / paper / 2020 / file / 6b493230205f780e1bc26945df7481e5-Paper. pdf.

Kwiatkowski, Tom, et al. "Natural Questions: A Benchmark for Question Answering Research". *Transactions of the Association for Computational Linguistics*, edited by Lillian Lee et al., vol. 7, 2019, pp. 452–66. https://doi.org/10.1162/tacl_a_00276.

Lewis, Mike, et al. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension". *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Edited by Dan Jurafsky et al., Association for Computational Linguistics, July 2020, pp. 7871–80, https://doi.org/10 . 18653 / v1 / 2020 . acl - main.703.

Bajaj, Payal, et al. MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. 2018. *arXiv*, arxiv . org / abs / 1611.09268.

Dunn, Matthew, et al. SearchQA: A New QA Dataset Augmented with Context from a Search Engine. 2017. *arXiv*, arxiv.org/abs/ 1704.05179.

Thorne, James, et al. "FEVER: a Large-scale Dataset for Fact Extraction and VERification". *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Edited by Marilyn Walker et al., Association for Computational Linguistics, June 2018, pp. 809–19, https://doi.org/10.18653/ v1/N18-1074.

Gao, Yunfan. "Retrieval-Augmented Generation for Large Language Models: A Survey". *arXiv preprint arXiv:2312.10997*, 2023.

Joren, Hailey, et al. "Sufficient Context: A New Lens on Retrieval Augmented Generation Systems". *International Conference on Learning Representations*. Edited by Y.

Table 2: Week-by-week project plan

Week	Phase	Julia	Nadia	Tomasz
1	Phase 1: Literature Review	Literature review on datasets, HotpotQA dataset analysis, literature review on MiniRAG, FlashRAG, and LongRAG, experimental pipeline design	Literature review on RAG and evaluation metrics, literature review on LangChain and LlamaIndex	Literature review on comparison of RAG frameworks
2	Phase 2: Implementation	HotpotQA preprocessing, exploratory data analysis, and relevance judgments	Implementation of LangChain and LlamaIndex retrieval pipelines	Implementation of MiniRAG, FlashRAG, and LongRAG retrieval pipelines
3	Phase 2: Proof of Concept	Evaluation scripts, baseline experiments, and dataset validation	Pipeline testing, initial retrieval results, and runtime measurements	Pipeline testing, initial retrieval results, and memory measurements
4	Phase 3: Final Evaluation	Final experiments, results collection, and statistical analysis	Framework comparison, error analysis, and final report preparation	Results visualization, reproducibility documentation, and final report preparation

Yue et al., 2025, pp. 20310–34, proceedings. iclr . cc / paper _ files / paper / 2025 / file / 33dffa2e3d2ab74a783d1a8c292f66d9-Paper-Conference.pdf.

Vu, Tu, et al. "FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation". *Findings of the Association for Computational Linguistics: ACL 2024*. Edited by Lun-Wei Ku et al., Association for Computational Linguistics, Aug. 2024, pp. 13697–720, https://doi.org/10 . 18653 / v1/2024.findings-acl.813.

Trivedi, Harsh, et al. " MuSiQue: Multihop Questions via Single-hop Question Composition". *Transactions of the Association for Computational Linguistics*, vol. 10, May 2022, pp. 539–54. *https://direct.mit.edu/tacl/article-pdf/doi/10.1162/tacl_a_00475/2020694/tacl_a_00475.pdf*, https://doi.org/10.1162/tacl_a_00475.

Yang, Zhilin, et al. "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering". *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2018.

Jin, Jiajie, et al. "FlashRAG: A Modular Toolkit for Efficient Retrieval-Augmented Generation Research". *Companion Proceedings of the ACM on Web Conference 2025*. WWW '25, Association for Computing Machinery, 2025, pp. 737–40, https://doi.org/10.1145/3701716.3715313.

Jiang, Ziyan, et al. LongRAG: Enhancing Retrieval-Augmented Generation with Long-context LLMs. 2024. *arXiv*, arxiv.org/abs/2406.15319.

Fan, Tianyu, et al. MiniRAG: Towards Extremely Simple Retrieval-Augmented Generation. 2025. *arXiv*, arxiv . org / abs / 2501 . 06713.

Chase, Harrison. *LangChain*, 17 Oct. 2022, github.com/langchain-ai/langchain.

Liu, Jerry. *LlamaIndex*, Nov. 2022, https://doi.org/10.5281/zenodo.1234.

Joshi, Mandar, et al. "TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension". *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Edited by Regina Barzilay and Min-Yen Kan, Association for Computational Linguistics, July 2017, pp. 1601–11, https://doi.org/10.18653/v1/P17-1147.

Es, Shahul, et al. "RAGAs: Automated Evaluation of Retrieval Augmented Generation". *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. Edited by Nikolaos Aletras and Orphee De Clercq, Association for Compu-

tational Linguistics, Mar. 2024, pp. 150–58, https://doi.org/10 . 18653 / v1 / 2024 . eacl - demo.16.

Thakur, Nandan, et al. "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models". *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. Edited by J. Vanschoren and S. Yeung, 2021, datasets - benchmarks - proceedings . neurips . cc / paper _ files / paper / 2021 / file / 65b9eea6e1cc6bb9f0cd2a47751a186f- Paper-round2.pdf.

Wang, Yining, et al. "A Theoretical Analysis of NDCG Type Ranking Measures". *Proceedings of the 26th Annual Conference on Learning Theory*. Edited by Shai Shalev-Shwartz and Ingo Steinwart, Proceedings of Machine Learning Research, PMLR, Dec. 2013, pp. 25–54, proceedings.mlr.press/v30/ Wang13.html.

Brown, Andrew, et al. "A Systematic Literature Review of Retrieval-Augmented Generation: Techniques, Metrics, and Challenges". *Big Data and Cognitive Computing*, vol. 9, no. 12, 2025, p. 320. https://doi.org/10 . 3390/bdcc9120320.

Appendix

A Contribution

Table 3: Contributions – Phase 1.

Team Member	Phase 1 Contributions	Hours
Julia	Literature review on datasets, HotpotQA dataset analysis, literature review on MiniRAG, FlashRAG, and LongRAG, experimental pipeline design	3
Nadia	Literature review on RAG and evaluation metrics, literature review on LangChain and LlamaIndex	3
Tomasz	Literature review on comparison of RAG frameworks	3

B AI usage

Generative AI was used for creating LaTeX tables, grammar enhancement of the report and creating a PUML file of the described experimental pipeline.